

课程笔记

CS224R Lecture 9: 面向 LLM 的强化学习与偏好优化

五道口纳什 & Codex

2025-04-30



视频作者/频道: Stanford Online

发布日期: 2025-12-08 (YouTube 上传)

视频时长: 1:02:51

视频链接: <https://www.youtube.com/watch?v=XKLGuwvSKvI>

目录

1 从语言模型到助手：问题究竟在哪里	2
1.1 预训练已经学到了很多，但还不等于“会帮助用户”	2
1.2 预训练/微调范式的基本形状	2
1.3 本章小结	2
2 Instruction Finetuning：先把模型变成会听指令的系统	3
2.1 为什么需要 instruction tuning	3
2.2 它的训练方式其实很简单	3
2.3 为什么 SFT 仍然不够	3
2.4 本章小结	3
3 RLHF：把人类偏好变成可优化目标	3
3.1 偏好监督的基本设定	3
3.2 reward model 与目标函数	4
3.3 为什么 reward model 可微，但仍然要做 RL	4
3.4 为什么要加 KL 约束	4
3.5 偏好数据本身也很吵	5
3.6 本章小结	5
4 DPO：直接从偏好数据优化策略	5
4.1 为什么要简化 RLHF	5
4.2 KL 约束目标的闭式解	5
4.3 DPO 的最终损失	5
4.4 DPO 与 RLHF 的关系	6
4.5 经验结果与社区影响	6
4.6 本章小结	7
5 局限性与未解问题	8
5.1 奖励模型和偏好都不是终点	8
5.2 数据收集与主动选择仍是难点	8
5.3 面向更复杂偏好的未来	8
5.4 本章小结	8
6 总结与延伸	9

1 从语言模型到助手：问题究竟在哪里

1.1 预训练已经学到了很多，但还不等于“会帮助用户”

讲者开场先回顾了过去几年最清晰的趋势：模型规模越来越大、算力投入越来越高，而这确实让语言模型学到了远超“语法续写”的能力。课程依次举了 trivia、句法、共指、基础推理、代码、医学知识等例子，说明 next-token prediction 在实践中会诱导出某种世界知识和任务能力。

但本讲要解决的并不是“模型有没有知识”，而是“这些知识如何变成助手行为”。预训练优化的是语料分布下的下一个 token，而不是用户真实意图。因此，一个会续写文本的模型，并不自动等价于一个会按要求解释、遵循指令、风格可控的 assistant。

1.2 预训练/微调范式的基本形状

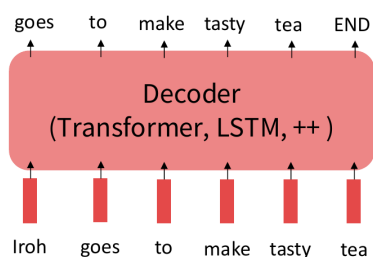
课程用一页非常经典的图来说明这种差异。预训练阶段大量吸收文本，学习一般性的语言与世界模式；微调阶段则在较小规模的标注数据上，让模型适配具体任务。

The Pretraining / Finetuning Paradigm

Pretraining can improve NLP applications by serving as parameter initialization.

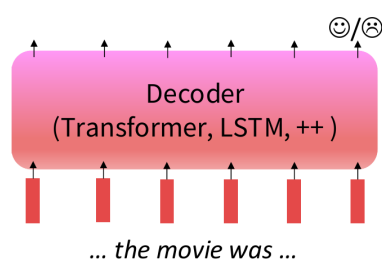
Step 1: Pretrain (on language modeling)

Lots of text; learn general things!



Step 2: Finetune (on your task)

Not many labels; adapt to the task!



41

图 1：预训练与微调范式：先学语言分布，再在具体任务标签上做适配。

这页图的重要性不只在提醒我们“先预训练再微调”这件事，而在于指出模型行为受目标函数支配。你教它优化什么，它就擅长什么；而用户想要的“帮助性、服从性、无害性”，在最初的预训练目标里并没有被直接表达。

1.3 本章小结

本讲的起点非常明确：大语言模型已经学会很多东西，但“知道很多”与“按人类意图行动”不是同一个问题。后训练的全部工作，都是在弥补这道缺口。

2 Instruction Finetuning: 先把模型变成会听指令的系统

2.1 为什么需要 instruction tuning

讲者用 GPT-3 式的反例说明，仅凭语言建模，模型经常只会继续写与提示词在统计上相邻的文本，而不是完成用户真正要求的任务。instruction finetuning (SFT) 就是把大量“指令-输出”样本对喂给模型，让它学会“把输入视作任务指令，而不是普通文本前缀”。

2.2 它的训练方式其实很简单

从目标函数上看，instruction tuning 并没有什么神秘之处。依然是条件语言建模，只不过训练数据从无标签互联网语料变成了高质量指令-回答样本。模型做的仍然是最大化参考输出的对数概率，但这些输出 now encode what users intended.

这种方法效果非常强，因为大模型一旦拥有足够基础能力，只要少量范例就能把“任务格式”对齐过来。课程里引用 Flan-T5 等工作，说明越大的预训练模型，在 instruction tuning 后往往获得越显著的跨任务泛化提升。

2.3 为什么 SFT 仍然不够

本讲明确指出了几类局限：

- 许多任务没有唯一正确答案，例如开放式创作。
- token-level log-likelihood 会把所有错误一视同仁，但人类在意的错误严重程度并不相同。
- 参考答案本身可能是次优的，模型会被教师质量所上限约束。
- 真正想优化的是“满足人类偏好”，而不是“复现单个参考答案”。

SFT 的角色

SFT 很像一个高质量初始化步骤。它把模型从“会续写文本”变成“基本会响应指令”，但它还没有显式表达偏好优化。

2.4 本章小结

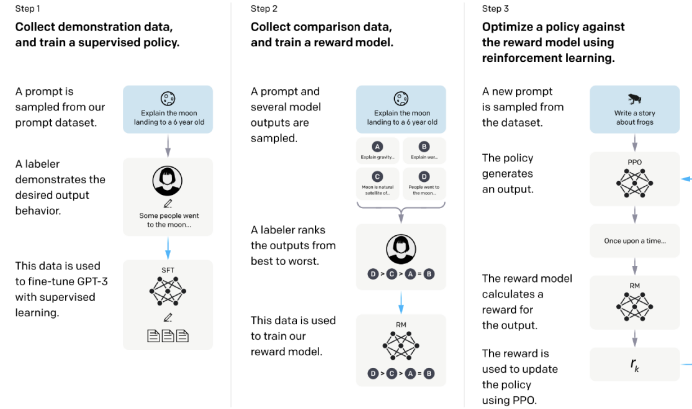
instruction tuning 解决了“把语言模型变成任务接口”的第一步，却没有真正解决“哪些回答更好、为什么更好”的偏好问题。

3 RLHF: 把人类偏好变成可优化目标

3.1 偏好监督的基本设定

当没有唯一标准答案时，人类通常仍然能比较两个回答谁更好。于是 RLHF 的经典设定是：给定 prompt x ，让模型采样若干候选回答，再由人类从中排序或做 pairwise comparison。课程强调，相比让人类给绝对分数，pairwise comparison 更稳定，也更符合标注实际。

High-level instantiation: ‘RLHF’ pipeline



- First step: instruction tuning!
- Second + third steps: maximize reward (but how??)

图 2: RLHF 高层流程: SFT 初始化, 收集比较数据训练 reward model, 再用 RL 优化策略。

3.2 reward model 与目标函数

如果用 $RM_\phi(x, y)$ 表示 reward model, 那么最理想的目标是直接优化

$$\max_{\theta} \mathbb{E}_{\hat{y} \sim p_{\theta}(\cdot|x)} [RM_\phi(x, \hat{y})].$$

对于偏好数据 (x, y^w, y^l) , reward model 本身通常可以通过 Bradley-Terry 风格的分类目标来训练:

$$J_{RM}(\phi) = -\mathbb{E}_{(x, y^w, y^l) \sim D} [\log \sigma(RM_\phi(x, y^w) - RM_\phi(x, y^l))].$$

这一步并不要求 reward model 给出“绝对真值”, 只要求它在排序上尽量与偏好标注一致。

3.3 为什么 reward model 可微, 但仍然要做 RL

讲者在问答里专门解释了一个常见疑问: 既然 reward model 往往也是神经网络、理论上可微, 为什么不能直接对生成模型反向传播, 而非使用 RL?

关键在于语言模型输出的是离散 token 序列。reward model 虽然对输入文本评分可微, 但生成过程包含离散采样, 这个离散瓶颈使得梯度无法稳定地直接穿回模型参数。因此需要借助 policy gradient 一类方法, 用

$$\nabla_{\theta} J(\theta) \approx \mathbb{E}_{\hat{y} \sim p_{\theta}(\cdot|x)} [RM_\phi(x, \hat{y}) \nabla_{\theta} \log p_{\theta}(\hat{y}|x)]$$

去优化期望奖励。

3.4 为什么要加 KL 约束

如果只追求 reward model 分数, 模型很快会学会利用 reward model 的漏洞。于是 RLHF 通常加入一个相对初始模型 p_{PT} 的 KL penalty:

$$\mathbb{E}_{\hat{y} \sim p_{\theta}(\cdot|x)} \left[RM_\phi(x, \hat{y}) - \beta \log \frac{p_{\theta}(\hat{y}|x)}{p_{PT}(\hat{y}|x)} \right].$$

它的作用不是装饰，而是把优化限制在 reward model 更可信的邻域里。课程里明确指出，reward model 本质上只是偏好数据上的近似器，因此策略不能偏离初始化分布太远。

RLHF 的真实难点

难点不是“会不会写 PPO”，而是 reward model 永远不完美。只要它有盲点，策略就会利用这些盲点。因此 RLHF 的稳定性高度依赖 KL 约束、采样分布和偏好数据质量。

3.5 偏好数据本身也很吵

讲者还提到一个经常被忽视的事实：人类偏好并不总是可传递的，数据也非常 noisy。即便 reward model 在这种二分类任务上只能达到大约 65% – 70% 的准确率，仍然足以显著改善模型行为。这一点很重要，因为它提醒我们：偏好优化依赖的是弱但有方向性的监督，而不是完美标签。

3.6 本章小结

RLHF 把“更受人类喜欢的回答”转化成可优化目标，核心组件是 reward model、policy gradient 和 KL 正则。它的成功不在于理论完美，而在于它第一次在大规模模型上把“偏好”真正接进了训练闭环。

4 DPO：直接从偏好数据优化策略

4.1 为什么要简化 RLHF

RLHF 在工业上有效，但实现复杂、算力开销大。讲者指出，大模型规模一旦很大，采样、训练 reward model、跑 RL、维护 value function 与各种工程细节都会迅速膨胀。因此一个非常自然的问题是：既然我们最终手里已经有偏好数据，为什么不能直接用它训练策略，而不显式经过完整 RL 环？

4.2 KL 约束目标的闭式解

关键观察是：带 KL 约束的奖励最大化目标有闭式最优策略

$$p^*(\hat{y}|x) = \frac{1}{Z(x)} p^{PT}(\hat{y}|x) \exp\left(\frac{1}{\beta} RM(x, \hat{y})\right).$$

把它重排后，可以把 reward model 写成

$$RM(x, \hat{y}) = \beta \log \frac{p^*(\hat{y}|x)}{p^{PT}(\hat{y}|x)} + \beta \log Z(x).$$

这一步的意义极大：reward 不再是完全独立的外部对象，而可以被写成“当前策略相对参考模型的对数比”。

4.3 DPO 的最终损失

再把上式代回偏好学习目标，只关心 winning 与 losing completion 的 reward 差，就会发现 $Z(x)$ 消掉了。于是得到 DPO 的核心损失：

$$J_{DPO}(\theta) = -\mathbb{E}_{(x, y^w, y^l) \sim D} \left[\log \sigma \left(\beta \log \frac{p_{\theta}(y^w|x)}{p_{PT}(y^w|x)} - \beta \log \frac{p_{\theta}(y^l|x)}{p_{PT}(y^l|x)} \right) \right].$$

这其实就是一个分类损失：提高 winning answer 在当前模型下的相对概率，降低 losing answer 的相对概率，同时以参考模型为基准控制漂移。

Direct Preference Optimization (DPO)

- Recall, how we fit the reward model $RM_\phi(x, y)$:

$$J_{RM}(\phi) = -\mathbb{E}_{(x, y^w, y^l) \sim D} [\log \sigma(RM_\phi(x, y^w) - RM_\phi(x, y^l))]$$

- Notice that we only need the **difference** between the rewards for y^w and y^l . Simplify for $RM_\theta(x, y)$:

$$RM_\theta(x, y^w) - RM_\theta(x, y^l) = \beta \log \frac{p_\theta^{RL}(y^w | x)}{p^{PT}(y^w | x)} - \beta \log \frac{p_\theta^{RL}(y^l | x)}{p^{PT}(y^l | x)}$$

- The final DPO loss function is:

$$J_{DPO}(\theta) = -\mathbb{E}_{(x, y^w, y^l) \sim D} [\log \sigma(RM_\theta(x, y^w) - RM_\theta(x, y^l))]$$

We have a *simple classification loss* function that connects **preference data** to **language model parameters** directly!

77

图 3: DPO 的核心推导：把奖励差写成策略相对参考模型的对数概率比，再直接用分类损失训练。

4.4 DPO 与 RLHF 的关系

讲者特别澄清了一点：DPO 并不是“完全不需要 reward model 思想”。相反，它是在数学上把 reward model 融进了策略参数化里。你仍然需要偏好数据，只是不再显式训练一个独立的 reward network 再跑 RL。于是工程更简单，训练也更稳定。

4.5 经验结果与社区影响

课程展示了 DPO 在 summarization 与 dialogue helpfulness 上和 PPO 级别方法相近甚至更强的结果，也强调了它在开源与闭源社区里的广泛采用。

Direct Preference Optimization (DPO)

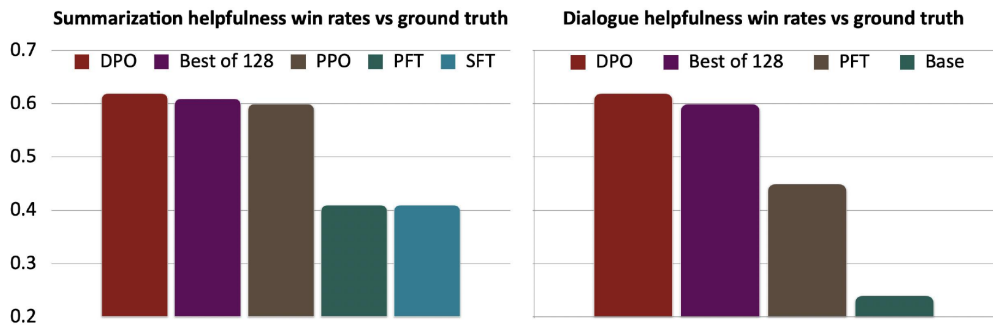
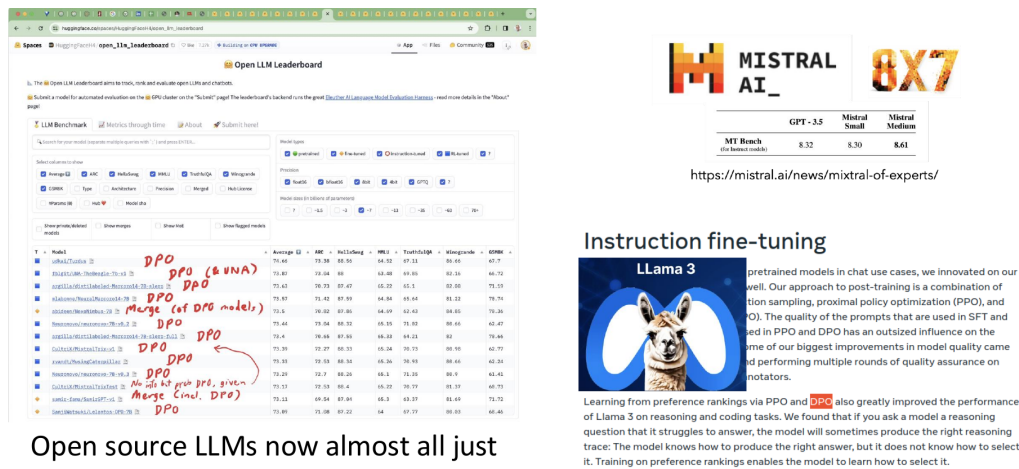


图 4: DPO 经验结果：直接偏好优化在 summarization 与 dialogue helpfulness 上具有强竞争力。

DPO is enabling open source and closed source models to improve!



Open source LLMs now almost all just use DPO (and it works well!)

87

图 5: DPO 的现实影响：它已经成为大量开源与闭源后训练流程中的标准组件。

4.6 本章小结

DPO 的核心贡献是把“偏好优化”从复杂 RL pipeline 重新表达为一个直接的分类目标。它保留了 RLHF 的偏好导向，同时显著降低了训练系统复杂度。

5 局限性与未解问题

5.1 奖励模型和偏好都不是终点

即便有 RLHF 或 DPO，问题也没有结束。课程最后集中讨论了几个深层局限。第一，人类偏好本身不稳定，群体偏好更是高度异质。第二，reward model 仍然可能被 exploit，产生 reward hacking。第三，pairwise preference 只能表达“哪个更好”，却很难充分表达“好多少”以及多目标之间的微妙权衡。

Limitations of RL + Reward Modeling

- Human preferences are unreliable!
- "Reward hacking" is a common problem in RL



<https://openai.com/blog/faulty-reward-functions/>

92

图 6: 奖励建模的局限：一旦 learned reward 有漏洞，优化过程就会出现 reward hacking。

5.2 数据收集与主动选择仍是难点

讲者还提到一个很值得注意的问题：如果偏好数据昂贵，是否应该主动挑选模型最困惑的输入去做人类比较，从而最大化信息增益？目前这仍然是开放方向。换句话说，我们不仅要研究“如何用偏好训练模型”，还要研究“该向人类询问哪些偏好最值钱”。

5.3 面向更复杂偏好的未来

课末的开放问题还包括：能否利用同一 prompt 下多样化采样得到更丰富的组合偏好；能否超越简单 pairwise comparison；企业是否会因 engagement 指标而学习出偏斜偏好；以及模型是否只会对“平均人类偏好”对齐，却忽视个体差异。这些都说明偏好优化不是一个纯技术性的小修小补，而是在触碰“系统究竟应该对谁负责”的更大问题。

5.4 本章小结

后训练方法已经很强，但“对齐”远未完成。今天的方法更多是在工程上找到了高效、稳定、可扩展的偏好优化近似，而不是彻底解决了人类价值建模。

6 总结与延伸

这节课把一条非常清晰的后训练主线串了起来。预训练让模型学会语言与世界知识, instruction tuning 把它变成一个基本会响应任务的系统, 而 RLHF / DPO 则进一步把人类偏好接入训练目标, 使模型不只会答题, 还会朝着“更符合人类期待”的方向移动。

如果用一句话概括 RLHF 与 DPO 的差异, 那么 RLHF 是“先学一个奖励模型, 再用 RL 优化策略”, 而 DPO 是“把带 KL 约束的奖励优化重写成直接的偏好分类训练”。两者都依赖偏好数据, 但 DPO 用更紧凑的形式把 reward learning 与策略优化合在了一起。

继续学习这条线时, 最值得追踪的不是某个具体 loss, 而是三个长期问题。第一, 偏好数据如何高效采集并减少噪声; 第二, 如何抑制 reward hacking 与分布外失真; 第三, 如何把群体偏好、个体偏好与安全约束同时纳入后训练过程。这些问题目前都没有定论, 而这也是“RL for LLMs”仍然是快速演化前沿的原因。